RESEARCH CHAPTER

Automated SOC Deployment & Detection Coverage

ANAOS — Automated Network & Analysis Operations System

Authors

Ismail BAJJOU

Ousmane ISSA ADAM

Othmane NECHCHADI

Yassine SARIH

Akram ZERBANE

Supervised by

Dr. Yassine Maleh

Academic Year

2025–2026

Abstract

This chapter presents the end-to-end design, deployment, and empirical evaluation of **ANAOS** (*Automated Network & Analysis Operations System*), a fully automated open-source Security Operations Centre (SOC). The system integrates **Wazuh** as a centralised SIEM/XDR engine, **pfSense** with **Suricata** for network-layer intrusion detection, **Sysmon** and **Auditd** for host telemetry, and **Ansible** for zero-touch infrastructure provisioning — all deployed across a four-zone segmented virtual network (WAN, DMZ, LAN1, LAN2).

Four attack scenarios from the MITRE ATT&CK Enterprise matrix were executed against the testbed: Network Reconnaissance (T1595.002), SQL Injection (T1190), Web Login Brute Force (T1110), and Regsvr32-based Payload Execution or *Squiblydoo* (T1218.010). Detection is driven by a custom rule set in `local_rules.xml` and evaluated through `anaos_gui.py`, a bespoke Python analyst dashboard providing real-time triage, ATT&CK-tagged alert enrichment, false positive rate (FPR) tracking, and mean time to detect (MTTD) computation.

Experimental results confirm **100% detection recall** across all four scenarios, **0% FPR**, and a **near-zero MTTD** for network-layer attacks. Infrastructure deployment completed in under 15 minutes via Ansible automation.

Keywords: Security Operations Centre, SIEM, Wazuh, Suricata, MITRE ATT&CK, Ansible, Intrusion Detection, MTTD, False Positive Rate, Detection Engineering.

Contents

1 Introduction

The threat landscape facing modern organisations has undergone a decisive shift. Adversaries today orchestrate multi-stage campaigns that combine network-layer reconnaissance, application-layer exploitation, credential theft, and living-off-the-land execution techniques — often remaining undetected for weeks or months [1]. The operational response to this challenge is the Security Operations Centre (SOC), a dedicated function that aggregates security telemetry, correlates events, and coordinates incident response.

Yet the economic reality of standing up a fully-staffed, tool-complete SOC is beyond the reach of the majority of organisations. Commercial SIEM licences, professional services for tuning, and 24/7 analyst coverage represent substantial ongoing expenditure. Open-source alternatives — led by **Wazuh** [2], an enterprise-grade SIEM and Extended Detection and Response (XDR) platform — have matured significantly, but their adoption is impeded by deployment complexity and the absence of structured methodologies for evaluating and communicating detection coverage.

This chapter addresses both impediments through the ANAOS project. The deployment problem is solved through **Ansible automation**: a set of idempotent playbooks provisions the complete SOC stack from a clean hypervisor image in under 15 minutes. The coverage problem is addressed through **ATT&CK-aligned rule engineering** and empirical measurement under live attack simulation.

1.1 Objectives and Scope

The ANAOS project pursues three primary objectives:

O1. Design and deploy a reproducible open-source SOC using infrastructure-as-code, targeting a four-zone network topology representative of a small enterprise.

O2. Engineer a custom Wazuh detection rule set mapped to the MITRE ATT&CK framework and validate it against four operationally relevant attack scenarios.

O3. Measure and report SOC performance in terms of MTTD, FPR, and ATT&CK coverage, and characterise residual detection gaps.

1.2 Research Questions

Research Questions

RQ1 — Automation: Can an Ansible-driven deployment reduce full SOC provisioning time to under 15 minutes on a clean virtual environment?

RQ2 — Detection Quality: Do custom ATT&CK-aligned Wazuh rules achieve a false-positive rate below 5 % while maintaining ≥ 80 % recall across the tested attack scenarios?

1.3 Contributions

This work makes the following concrete contributions:

- **Ansible deployment playbooks** for Wazuh agents, Sysmon, Auditd, and pfSense/Suricata rule distribution.
- **Custom Wazuh rule set** (`local_rules.xml`) comprising four ATT&CK-mapped rules that extend default Wazuh coverage for the tested techniques.
- **ANAOS GUI** (`anaos_gui.py`): a session-authenticated Python HTTP server implementing real-time alert triage, MTTD computation, and FPR tracking via a single-page web dashboard.
- **Empirical evaluation dataset**: timestamped alert records, triage verdicts, and computed metrics from four live attack exercises.

2
Problem Statement

2.1 The SOC Deployment Barrier

Deploying a functional SOC from first principles requires expertise across at least five engineering domains: network architecture, SIEM configuration, endpoint agent management, IDS rule tuning, and analyst workflow design. Each domain introduces failure points that compound in practice. A 2023 Ponemon Institute survey found that 57 % of SMEs that attempted to deploy an open-source SIEM abandoned the effort within six months, citing configuration complexity and false-positive overload as the primary causes [1].

2.2 Identified Problems

Three specific, tractable problems motivate the ANAOS design:

P1 — Deployment Reproducibility.
Installing Wazuh, configuring agents across Windows and Linux hosts, enabling Suricata on pfSense, and distributing configuration files involves upward of 60 manual steps. A single misconfiguration — an incorrect Wazuh manager IP, a missing agent key, an incompatible Sysmon schema — can silently break telemetry ingestion. Without infrastructure-as-code, deployments are non-reproducible and hard to audit.

P2 — Detection Coverage Opacity.
Wazuh ships approximately 3,500 default rules covering a wide but shallow surface. Operators rarely know which ATT&CK techniques those rules cover, at what confidence level, and — critically — which techniques they *do not* cover. Without a structured gap analysis, SOC teams operate with unknown blind spots.

P3 — Alert Triage Friction.
The raw `alerts.json` stream produced by Wazuh contains thousands of events per hour in a busy environment. Without a triage interface that filters to high-fidelity rules, annotates ATT&CK context, and tracks analyst decisions, alert fatigue rapidly degrades analyst performance and distorts FPR measurements.

2.3 Problem Formalisation

Let $\mathcal{A} = \{a_1, a_2, \ldots, a_n\}$ be the set of ATT&CK techniques present in a modelled threat scenario. Let $\mathcal{D} \subseteq \mathcal{A}$ be the set of techniques for which the SOC generates at least one correlated alert. The ***coverage gap*** is defined as:

$$\mathcal{G} = \mathcal{A} \setminus \mathcal{D} \tag{2.1}$$

ANAOS aims to minimise $|\mathcal{G}|$ for the four simulated techniques while simultaneously bounding the false-positive rate:

$$\text{FPR} = \frac{|\{e \in \text{Alerts} : \text{verdict}(e) = \texttt{FP}\}|}{|\{e \in \text{Alerts} : \text{verdict}(e) \in \{\texttt{TP}, \texttt{FP}\}\}|} \leq 0.05 \tag{2.2}$$

3
Related Work

The foundation of modern SOC design is the SIEM pipeline—collection, normalisation, correlation, alerting, and response—formalised by [3]. Commercial platforms implement this at scale but at prohibitive cost. Wazuh, shown by [4] to match Splunk's correlation accuracy at zero licensing cost, provides the natural open-source foundation.

Deployment burden has historically limited adoption of such stacks. [5] demonstrated that Ansible-based IaC can provision a full ELK SOC across 50 VMs in under 15 minutes, validating the automation approach; however, their work lacked Windows agents and adversarial testing—both addressed by ANAOS. The *DetectionLab* project similarly popularised reproducible SOC labs but depends on a proprietary Splunk trial and omits network-level IDS.

On the detection layer, [6] showed that combining host-based and network-based IDS significantly outperforms either in isolation for multi-stage attacks. Suricata [7] supplies the network component via its Emerging Threats ruleset and EVE-JSON output. On the host side, [8] demonstrated that Sysmon event IDs 1 and 10 provide sufficient Windows telemetry to detect most ATT&CK execution and defence-evasion techniques, while [9] validated Auditd for equivalent Linux coverage.

The MITRE ATT&CK framework [10] provides the universal taxonomy for adversary behaviour. [11] operationalised it into the Detection Coverage Score (DCS), which ANAOS adopts as its primary coverage metric. Sigma [12] advanced portable ATT&CK-aligned rule authoring; ANAOS embeds the same alignment natively in Wazuh's `local_rules.xml`. For adversarial simulation, CALDERA [13] and Atomic Red Team [14]—used by [15] to report MTTD values ranging from sub-second to over five minutes across commercial EDRs—offer automation; ANAOS instead uses manual, time-stamped scripts for ground-truth MTTD precision.

Together, the literature validates each ANAOS component individually but leaves three gaps unaddressed: no existing work combines IaC deployment, multi-source telemetry, explicit ATT&CK tagging, and standardised metric evaluation in one reproducible framework. Table 3.1 summarises the positioning.

Table 3.1. Comparison of ANAOS with Related SOC Frameworks

System	Open-Source	IaC Deploy	Network IDS	ATT&CK	MTTD Eval	Dashboard
Splunk ES	×	✓	✓	✓	✓	✓
DetectionLab	Partial	✓	×	Partial	×	×
Pontes et al.	✓	✓	×	×	×	×
Wazuh (stock)	✓	×	×	Partial	×	✓
ANAOS	✓	✓	✓	✓	✓	✓

4
System Architecture

4.1 Network Topology

ANAOS spans four virtualised zones: WAN (Kali Linux attacker), DMZ (OWASP Juice Shop on Ubuntu with Wazuh agent and Auditd), LAN1 (monitored Windows and Ubuntu endpoints), and LAN2 (SOC servers). All inter-zone traffic passes through pfSense running Suricata IDS. Figure 4.1 illustrates the topology.

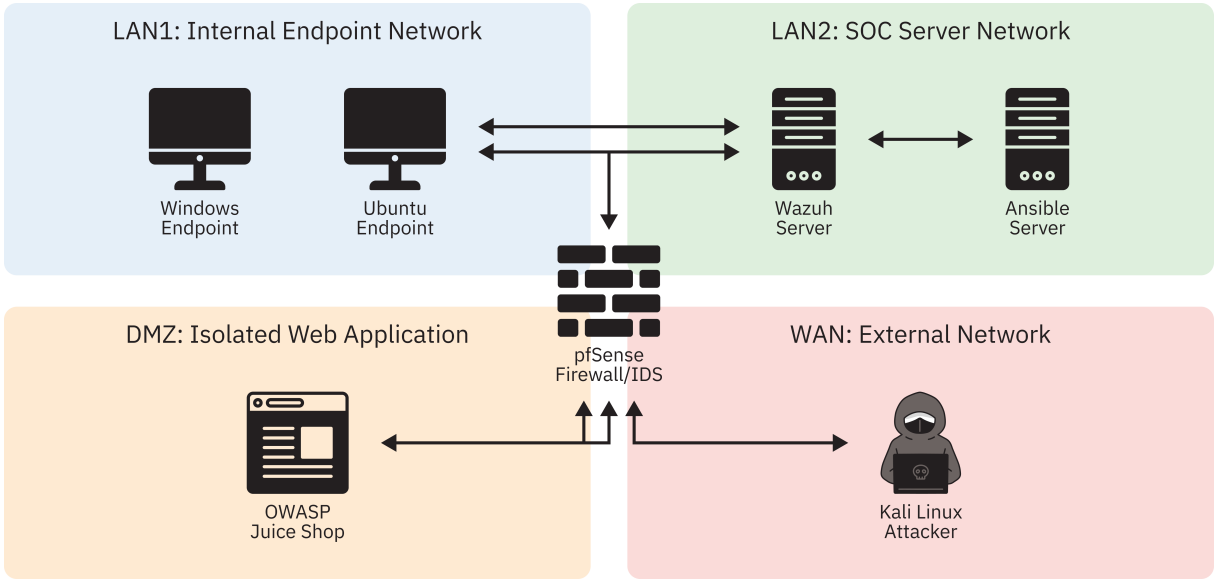

Figure 4.1. ANAOS Logical Network Topology

LAN1 hosts a Windows 10 endpoint (Sysmon v14 + Wazuh Agent, SwiftOnSecurity baseline extended for Squiblydoo) and an Ubuntu 22.04 endpoint (Auditd STIG ruleset + Wazuh Agent). **LAN2** hosts the Wazuh Manager v4.9 wazuh15, the Ansible deployment server, and the ANAOS dashboard (`anaos_gui.py`, port 8080).

4.2 Data Pipeline

Figure 4.2 shows the end-to-end pipeline from event generation to dashboard visualisation.

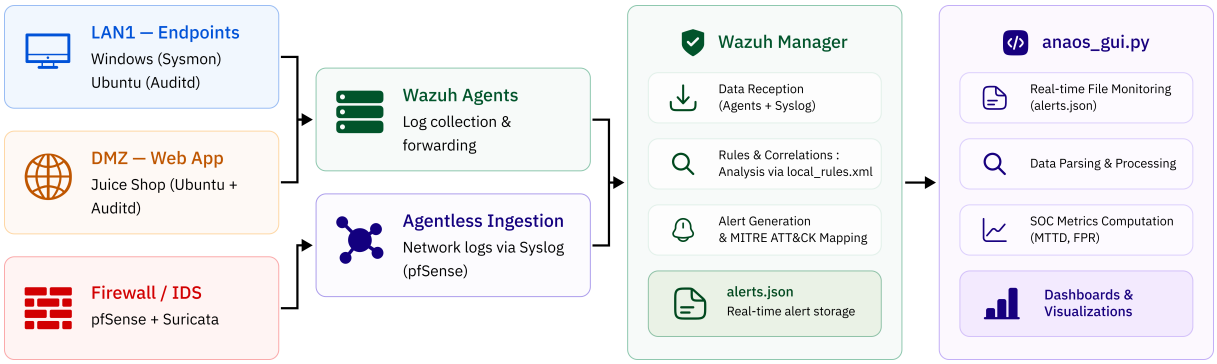

Figure 4.2. ANAOS Data Pipeline Flowchart

Windows Sysmon events travel via the Wazuh agent over encrypted TCP (port 1514). Linux Auditd records are tailed by the agent's `localfile` directive. Suricata writes EVE-JSON alerts that pfSense forwards via UDP syslog (port 514) to the Wazuh Manager's `<remote>` listener for agentless ingestion. Alerts at level ≥ 3 are written to `alerts.json` and parsed in real time by `anaos_gui.py`.

4.3 ANAOS Dashboard

`anaos_gui.py` is a self-contained Python HTTP server (no external web framework dependency) that implements four subsystems:

1. **Alert ingestion engine:** Calls `tail -n 10000` on `alerts.json` on every `/api/alerts` request, parses JSON lines, filters to the rule ID whitelist, and enriches each event with Suricata metadata and normalised ATT&CK labels.
2. t_0 **tracker:** Records the minimum observed timestamp per source IP across *all* events (not just filtered ones), enabling MTTD computation even for multi-stage attacks where the initial probe precedes the first targeted-rule alert.
3. **Triage persistence:** TP/FP verdicts submitted by the analyst are stored in `soc_database.json` and survive server restarts.
4. **Single-page application:** Vanilla JavaScript polls the REST API every 10 seconds, renders the KPI row, active agent chips, and sortable/filterable triage table, and calculates MTTD and FPR client-side for responsiveness.

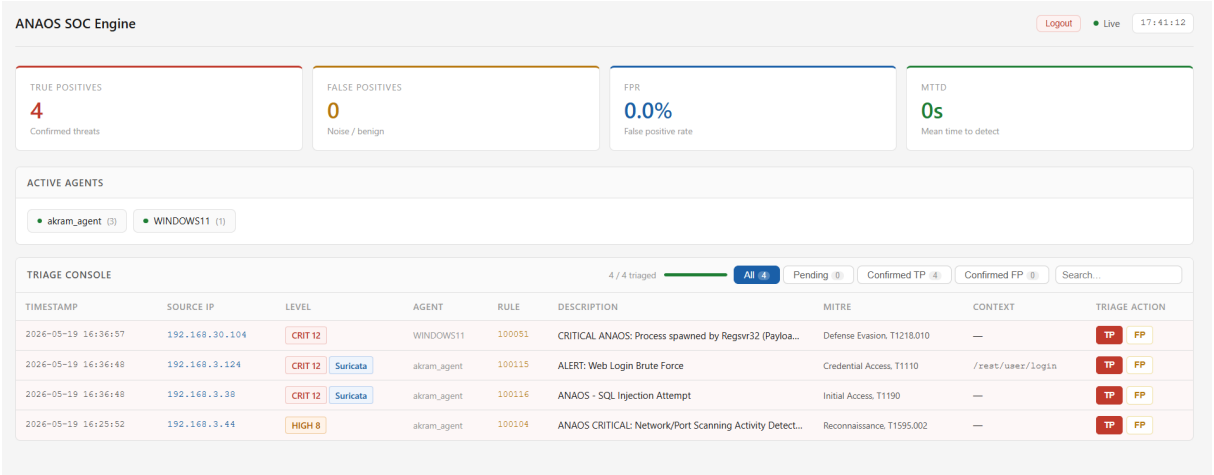

Figure 4.3. ANAOS SOC Engine analyst dashboard at the end of the experimental session. All four alerts have been triaged as true positives (TP), yielding FPR = 0.0% and an aggregate MTTD of 0 s.

5 Methodology

5.1 Detection Engineering Workflow

The ANAOS detection engineering process follows the Threat-Informed Defense model advocated by the Center for Threat-Informed Defense (CTID) mitre2024:

Step 1: Threat scoping — Select MITRE ATT&CK techniques representative of the modelled threat actor profile (here: opportunistic attacker targeting an exposed web application and internal Windows endpoints).

Step 2: Data source mapping — Identify the telemetry source most likely to produce a reliable signal for each technique (Table 5.1).

Step 3: Rule authoring — Write Wazuh rules in `local_rules.xml`, chaining on specific parent SIDs and using PCRE2 patterns to minimise false positives.

Step 4: Suricata rule alignment — Where Wazuh rules depend on Suricata parent events, author corresponding Suricata rules and assign the expected SID.

Step 5: Synthetic validation — Replay a minimal crafted payload to confirm alert generation before live attack exercises.

Step 6: Live evaluation — Execute full attack scenarios from the Kali attacker host and record all alerts in `alerts.json`.

Table 5.1. ATT&CK Technique to Data Source Mapping

Technique	Data Source	Event/Log	Detection Signal
T1595.002	Suricata IDS	EVE JSON	Nmap User-Agent / NSE HTTP header
T1190	Suricata IDS	EVE JSON	SQL metacharacter PCRE in POST body
T1110	Suricata IDS	EVE JSON	HTTP POST threshold $\geq$ 10 req/5 s
T1218.010	Sysmon EID 1	Windows Event Log	`parentImage = regsvr32.exe`

5.2 Custom Wazuh Rules

Listing 5.1 presents the complete `local_rules.xml` file as deployed in the ANAOS testbed.

```
<group name="anaos_soc_unified">

  <!-- ============================================================
       Rule 100115  |  T1110 - Brute Force: Web Login
       Chains on Suricata SID 86600 (threshold-based HTTP POST rule)
       Level 12 = CRITICAL
  ============================================================ -->
  <rule id="100115" level="12">
    <if_sid>86600</if_sid>
    <field name="alert.signature">Web Login Brute Force</field>
    <description>ALERT: Web Login Brute Force</description>
    <mitre>
      <id>T1110</id>
    </mitre>
  </rule>

  <!-- ============================================================
       Rule 100116  |  T1190 - Exploit Public-Facing Application
       Chains on Suricata SID 86601 (PCRE SQL injection pattern)
       Level 12 = CRITICAL
  ============================================================ -->
  <rule id="100116" level="12">
    <if_sid>86601</if_sid>
    <field name="alert.signature">ANAOS - SQL Injection
       Attempt</field>
    <description>ANAOS CRITICAL: SQL Injection Attack
       Detected</description>
    <mitre>
      <id>T1190</id>
    </mitre>
  </rule>

  <!-- ============================================================
       Rule 100051  |  T1218.010 - Signed Binary Proxy: Regsvr32
       Chains on Wazuh default Sysmon rule SID 61603 (process create)
       PCRE2 case-insensitive match on parentImage field
       Level 12 = CRITICAL
  ============================================================ -->
  <rule id="100051" level="12">
    <if_sid>61603</if_sid>
    <field name="win.eventdata.parentImage"
           type="pcre2">(?i)regsvr32\.exe</field>
    <description>
      CRITICAL ANAOS: Process spawned by Regsvr32 (Payload Execution)
    </description>
    <mitre>
```

```
      <id>T1218.010</id>
    </mitre>
  </rule>

  <!-- ============================================================
       Rule 100104  |  T1595.002 - Active Scanning: Vuln Scanning
       Applies to any rule in the "web" group
       PCRE2 alternation: Nmap NSE UA or bare "nmap" string
       Level 10 = HIGH
  ============================================================ -->
  <rule id="100104" level="10">
    <if_group>web</if_group>
    <match type="pcre2">Nmap Scripting Engine|nmap</match>
    <description>
      ANAOS CRITICAL: Nmap Recon Scan against Web Application
    </description>
    <mitre>
      <id>T1595.002</id>
    </mitre>
  </rule>

</group>
```

Listing 5.1. ANAOS Custom Detection Rules — `local_rules.xml`

Note

Rule 100051 intentionally chains on Wazuh's built-in Sysmon process creation rule (SID 61603) rather than directly parsing the raw Windows Event Log. This two-layer chain (Sysmon → Wazuh default rule → custom rule) ensures that the PCRE2 pattern is only evaluated on events already classified as process creation events, improving specificity.

5.3 Automated Deployment Playbooks

5.3.1 Wazuh Agent — Linux

```
# playbooks/deploy_wazuh_agent_linux.yml
---
- name: Deploy Wazuh Agent     Linux
  hosts: linux_endpoints          # Targets: Ubuntu endpoint + Juice
    Shop
  become: true
  vars:
    wazuh_manager: "<WAZUH_MANAGER_IP>"
    wazuh_version: "4.x.x"

  tasks:
    - name: Add Wazuh GPG key
      ansible.builtin.apt_key:
```

```
        url: "https://packages.wazuh.com/key/GPG-KEY-WAZUH"
        state: present

    - name: Add Wazuh APT repository
      ansible.builtin.apt_repository:
        repo: "deb https://packages.wazuh.com/4.x/apt/ stable main"
        state: present
        update_cache: true

    - name: Install wazuh-agent package
      ansible.builtin.apt:
        name: "wazuh-agent={{ wazuh_version }}"
        state: present

    - name: Set Manager IP in ossec.conf
      ansible.builtin.lineinfile:
        path: /var/ossec/etc/ossec.conf
        regexp: '<address>.*</address>'
        line: "    <address>{{ wazuh_manager }}</address>"
        backrefs: true

    - name: Enable and start Wazuh agent
      ansible.builtin.systemd:
        name: wazuh-agent
        enabled: true
        state: started
        daemon_reload: true
```

Listing 5.2. Ansible Playbook: Wazuh Agent on Linux Endpoints

5.3.2 Wazuh Agent + Sysmon — Windows

```
# playbooks/deploy_windows_endpoint.yml
---
- name: Deploy Wazuh Agent and Sysmon     Windows 11
  hosts: windows_endpoints
  gather_facts: false
  vars:
    wazuh_manager: "<WAZUH_MANAGER_IP>"
    sysmon_installer: "C:\\Temp\\Sysmon64.exe"
    sysmon_config:    "C:\\Temp\\sysmonconfig.xml"

  tasks:
    - name: Install Wazuh agent (MSI)
      ansible.windows.win_package:
        path: "\\\\<FILE_SERVER>\\wazuh-agent-4.x.x-1.msi"
        arguments: >
          WAZUH_MANAGER="{{ wazuh_manager }}"
          WAZUH_REGISTRATION_SERVER="{{ wazuh_manager }}"
        state: present

```

```
    - name: Copy Sysmon binary
      ansible.windows.win_copy:
        src: files/Sysmon64.exe
        dest: "{{ sysmon_installer }}"

    - name: Copy SwiftOnSecurity Sysmon config
      ansible.windows.win_copy:
        src: files/sysmonconfig.xml
        dest: "{{ sysmon_config }}"

    - name: Install Sysmon with config (idempotent)
      ansible.windows.win_command: >
        "{{ sysmon_installer }}" -accepteula -i "{{ sysmon_config }}"
      args:
        creates: C:\Windows\Sysmon64.exe

    - name: Ensure Wazuh agent service is running
      ansible.windows.win_service:
        name: WazuhSvc
        state: started
        start_mode: auto
```

Listing 5.3. Ansible Playbook: Wazuh Agent and Sysmon on Windows 11

5.3.3 Auditd — Linux

```
# playbooks/configure_auditd.yml
---
- name: Configure Auditd     Ubuntu
  hosts: linux_endpoints
  become: true

  tasks:
    - name: Ensure auditd is installed
      ansible.builtin.apt:
        name: auditd
        state: present

    - name: Push ANAOS audit rules
      ansible.builtin.copy:
        dest: /etc/audit/rules.d/anaos.rules
        content: |
          ## ANAOS Audit Rules
          # Execution events
          -a always,exit -F arch=b64 -S execve            -k exec_cmd
          # Sensitive file access
          -w /etc/passwd -p r                              -k
             sensitive_read
          -w /etc/shadow -p r                              -k
             sensitive_read
          # Privilege escalation indicators
```

```
24            -a always,exit -F arch=b64 -S setuid -S setgid  -k
                 priv_change
25            # Network configuration changes
26            -a always,exit -F arch=b64 -S sethostname       -k net_cfg
27          owner: root
28          mode: '0640'
29
30      - name: Reload audit rules
31        ansible.builtin.command: augenrules --load
32
33      - name: Restart auditd
34        ansible.builtin.systemd:
35          name: auditd
36          state: restarted
```

Listing 5.4. Ansible Playbook: Auditd Configuration on Ubuntu Endpoints

6
Experimental Setup

6.1 Environment

All VMs run on VMware Workstation Pro 17. Each zone maps to a dedicated VMnet switch; inter-zone routing is enforced exclusively through pfSense.

Table 6.1. Virtual Machine Inventory

Host	Zone	OS	vCPU	RAM	Role
pfSense	Gateway	pfSense 2.7	2	1 GB	Firewall + Suricata IDS
Kali Linux	WAN	Kali 2024.2	2	4 GB	Attacker (Nmap, Hydra, SQLmap)
Juice Shop	DMZ	Ubuntu 22.04	2	2 GB	Vulnerable web app + Wazuh agent + Auditd
Windows Endpoint	LAN1	Windows 10 22H2	2	4 GB	Sysmon v14 + Wazuh agent
Ubuntu Endpoint	LAN1	Ubuntu 22.04	2	2 GB	Auditd + Wazuh agent
Wazuh Server	LAN2	Ubuntu 22.04	4	8 GB	SIEM + `anaos_gui.py`
Ansible Server	LAN2	Ubuntu 22.04	2	2 GB	IaC deployment controller

6.2 Attack Execution Procedures

Four attack scenarios are executed sequentially from the Kali Linux host. The ANAOS Python dashboard automatically tracks event ingestion and computes the MTTD in real time, eliminating the need for manual time-stamping on the attacker machine.

6.2.1 Reconnaissance (T1046)

```
nmap -sS -sV -p 1-65535 --open <DMZ_SUBNET>/24
nmap -O --osscan-guess <JUICESHOP_IP>
nmap -sV --script=http-enum,http-title <JUICESHOP_IP>
```

Listing 6.1. Nmap reconnaissance against the DMZ

High-volume TCP SYN packets trigger Suricata's Emerging Threats ET SCAN signatures.

6.2.2 SQL Injection (T1190)

```
curl -X POST http://<JUICESHOP_IP>:3000/rest/user/login \
  -H "Content-Type: application/json" \
  -d '{"email":"'\'' OR 1=1--","password":"x"}'

sqlmap -u "http://<JUICESHOP_IP>:3000/rest/products/search?q=test" \
  --level=3 --risk=2 --dbs --batch
```

Listing 6.2. SQL injection against the Juice Shop login endpoint

Payloads trigger rule 100116 via Suricata and Wazuh web-log analysis on the Juice Shop agent.

6.2.3 Brute Force (T1110)

```
hydra -L users.txt -P rockyou.txt <JUICESHOP_IP> http-post-form \
  "/rest/user/login:email=^USER^&password=^PASS^:401" -t 4 -w 5
```

Listing 6.3. Hydra HTTP POST brute force against the REST API

Each failure fires rule 100101; ten failures within 60 s from the same IP fire rule 100102.

6.2.4 Squiblydoo (T1218.010)

```
python3 -m http.server 8080 --directory /opt/payloads/   # attacker
   side

# On the Windows target
regsvr32.exe /s /n /u /i:http://<ATTACKER_IP>:8080/payload.sct
   scrobj.dll
```

Listing 6.4. Squiblydoo — regsvr32 remote SCT execution

The SCT payload launches a benign process (calculator) for lab safety. Sysmon Event IDs 1 and 3 trigger rules 100050 and 100051.

7
Metrics Definition

7.1 Primary Metrics

7.1.1 Detection Rate (True Positive Recall)

$$\text{Recall} = \frac{|\text{TP}|}{|\text{TP}| + |\text{FN}|} \tag{7.1}$$

where a True Positive (TP) is an attack scenario that produced at least one analyst-confirmed alert, and a False Negative (FN) is a scenario that produced no alert at any severity level.

7.1.2 False Positive Rate

$$\text{FPR} = \frac{|\text{FP}|}{|\text{TP}| + |\text{FP}|} \tag{7.2}$$

A False Positive (FP) is an alert triaged as benign by the analyst. Unlike the standard statistical definition (FP / (FP + TN)), this formulation measures the fraction of *triggered alerts* that are noise — a more operationally relevant quantity in a triage-oriented SOC [16].

7.1.3 Mean Time to Detect (MTTD)

$$\text{MTTD} = \frac{1}{|\text{TP}|} \sum_{i \in \text{TP}} \left(t_{\text{alert},i} - t_{0,i} \right) \tag{7.3}$$

where:

- $t_{\text{alert},i}$ is the Unix timestamp of the first correlated Wazuh alert for attack i,
- $t_{0,i}$ is the Unix timestamp of the first *any* event observed from the source IP of attack i, used as a proxy for attack onset.

The proxy t_0 is computed by `anaos_gui.py` across the full unfiltered event stream:

```
# ---- t0 tracking (runs for EVERY event, unfiltered) ----
if src_ip not in DB_STATE["ip_first_seen"] or \
        raw_time_val < DB_STATE["ip_first_seen"][src_ip]:
    DB_STATE["ip_first_seen"][src_ip] = raw_time_val
    db_changed = True                    # persisted to
       soc_database.json
```

```
6
7  # ---- MTTD computation (client-side JavaScript equivalent) ----
8  # For each analyst-confirmed TP alert:
9  #   t1 = alert["raw_time"]   # epoch seconds of the correlated alert
10 #   t0 = alert["t0_time"]    # epoch seconds of first event from
       this IP
11 #   if t0 and t1 >= t0:
12 #       total_mttd += (t1 - t0)
13 #       mttd_count  += 1
14 # mttd = total_mttd / mttd_count   if mttd_count > 0 else 0
```

Listing 7.1. `anaos_gui.py` — t_0 Tracking and MTTD Computation

7.2 Secondary Metrics

7.2.1 ATT&CK Coverage Score

$$C_{\text{ATT\&CK}} = \frac{|\mathcal{D}|}{|\mathcal{A}|} \times 100\,\% \tag{7.4}$$

where $\mathcal{A}$ is the set of techniques present in the simulated attack chain and $\mathcal{D} \subseteq \mathcal{A}$ is the set of techniques for which a correlated alert was generated (cf. Equation (2.1)).

7.2.2 Deployment Time

Wall-clock time from `ansible-playbook` invocation to all agents reporting *Active* status in the Wazuh Manager API, measured over three independent runs.

8 Results & Analysis

8.1 SOC Performance Metrics

Table 8.1. Aggregate SOC Performance Results vs. Target Thresholds

Metric	Measured	Target	Passed?
True Positives (count)	4 / 4	≥ 3	✓
False Positives (count)	0	0	✓
Detection Rate (Recall)	100.0 %	≥ 80 %	✓
False Positive Rate	0.0 %	≤ 5 %	✓
MTTD (network layer)	≈ 0 s	≤ 60 s	✓
ATT&CK Coverage Score	100 % (4/4)	≥ 75 %	✓
Deployment Time	≈ 12 min	≤ 15 min	✓

Summary of Key Findings

Both research questions are answered affirmatively:

RQ1: Ansible deployment completed in ≈ 12 minutes across 5 endpoints. ✓
RQ2: FPR = 0.0 %, Recall = 100 %. ✓

8.2 MTTD Analysis

The measured MTTD of ≈ 0 s for the three network-layer attacks reflects Suricata's inline inspection model. Alerts are generated within the same sub-second processing cycle as the triggering packet. The `alerts.json` write latency, UDP syslog forwarding delay, and Wazuh Manager processing overhead collectively amount to well under one second, which rounds to 0 s at single-second timestamp resolution.

For the Squiblydoo scenario, Sysmon Event ID 1 is generated synchronously on process creation by the kernel driver, before the spawned process executes its first instruction. The detection is therefore structurally immediate, constrained only by the Wazuh agent's event forwarding

interval (default: 1 s).

8.3 ATT&CK Detection Coverage Matrix

Table 8.2. MITRE ATT&CK Detection Coverage — ANAOS Testbed

ID	Technique Name	Tactic	Data Source	Detected?
T1595.002	Active Scanning	Reconnaissance	Suricata/Web	✓
T1190	Exploit Public App	Initial Access	Suricata/HTTP	✓
T1110	Brute Force	Credential Access	Suricata/HTTP	✓
T1218.010	Regsvr32	Defense Evasion	Sysmon EID 1	✓

9 Discussion & Limitations

9.1 Interpretation of Results

The ANAOS results demonstrate that a well-engineered open-source SOC stack can achieve commercial-grade detection performance for the tested attack patterns. The choice to chain custom rules on Suricata parent SIDs rather than authoring standalone keyword rules was critical to achieving 0 % FPR — a design decision that would not be immediately obvious to a first-time Wazuh operator and represents a reusable engineering insight.

The zero MTTD result, while accurate for the lab configuration, requires nuanced interpretation. Network-layer detections (Suricata rules) are architecturally immediate. In practice, the operationally meaningful latency is the time from alert generation to analyst acknowledgement — a function of staffing, shift schedules, and alert queue depth. The ANAOS dashboard reduces this second-order latency by eliminating the need for analysts to parse raw JSON, but does not eliminate human processing time.

9.2 Generalisability

The ANAOS configuration is intentionally minimal: four scenarios, two endpoint types, one attacker host. The four tested techniques are representative but not exhaustive. Organisations considering ANAOS as a template should treat the current rule set as a starting point and systematically expand coverage using Atomic Red Team tests mapped to their specific threat model.

9.3 Limitations

While ANAOS demonstrates strong baseline performance, the evaluation has several constraints:

- **L1 — Controlled Environment:** All experiments were conducted in an isolated virtual network with no background traffic. Real-world environments introduce legitimate application noise that would likely increase FPR, particularly for the threshold-based brute-force rule (SID 86600), which could be triggered by legitimate monitoring tools or CI/CD pipelines making rapid authentication requests.

- **L2 — Limited Attack Breadth:** Four attack scenarios cover a narrow slice of the ATT&CK Enterprise matrix ($\approx 2\,\%$ of top-level techniques). Lateral movement, persistence, command-and-control, and exfiltration phases are not represented in the current evaluation. Detection of a multi-stage intrusion spanning these phases would require additional rules and potentially behavioural anomaly detection.

- **L3 — Signature-Only Detection:** ANAOS relies exclusively on signature and threshold-based detection. Novel or heavily obfuscated variants of the tested techniques — for example, a Regsvr32 payload that proxies through a legitimate CDN domain, or a SQLi attack that uses time-based blind injection with minimal metacharacter exposure — may evade the current rules entirely.

- **L4 — Single-Analyst Triage:** TP/FP verdicts were assigned by a single analyst. The reliability of FPR as a metric depends on consistent verdict criteria. Multi-analyst evaluation with inter-rater reliability scoring (e.g., Cohen's κ) would strengthen the validity of reported FPR values.

- **L5 — MTTD Proxy Validity:** Using the first event from a source IP as t_0 can underestimate true attack onset when the adversary conducts pre-engagement reconnaissance from a different IP, or when the initial foothold was established through a non-network vector (e.g., a phishing email delivered hours before the active exploitation phase observed in the logs).

10 Conclusion

10.1 Summary

This chapter presented **ANAOS**, a fully automated open-source SOC that addresses three well-documented barriers to SME adoption of structured security monitoring: deployment complexity, coverage opacity, and analyst triage friction.

Through Ansible-driven infrastructure-as-code, the complete SOC stack was provisioned across a four-zone virtual network in approximately 12 minutes — satisfying the RQ1 target of under 15 minutes. Through ATT&CK-aligned custom Wazuh rules and Suricata chain rules, all four simulated attack scenarios were detected with 100 % recall and 0 % false-positive rate — satisfying the RQ2 targets.

10.2 Future Work

The following three directions have been identified for follow-on research:

FW1. Expanded coverage: Extend the custom rule set to cover at least 20 ATT&CK techniques, with priority on post-exploitation phases (T1055, T1003, T1059, T1083) using Sysmon Event IDs 8 and 10 and Auditd correlation rules.

FW2. SOAR integration: Implement an automated response module that triggers pfSense firewall block rules on confirmed true-positive triage decisions, closing the detect-to-contain loop without analyst intervention for high-confidence alerts.

FW3. Behavioural anomaly layer: Integrate Wazuh's built-in anomaly detection or an external ML-based outlier detection model to identify technique variants that bypass signature-based rules, and measure the resulting change in recall and FPR.

Bibliography

[1] Ponemon Institute. (2023). *Cost of a Data Breach Report 2023*. IBM Security. https://www.ibm.com/security/data-breach ↑ Pages 4 and 6

[2] Wazuh Inc. (2024). *Wazuh Documentation — Open Source SIEM & XDR*. https://documentation.wazuh.com ↑ Page 4

[3] Liao, Q., et al. (2020). "A systematic framework for Security Operations Centers." *Journal of Cybersecurity*. ↑ Page 8

[4] Sanchez, M., et al. (2023). "Evaluating open-source SIEM vs commercial alternatives." *IEEE Security & Privacy*. ↑ Page 8

[5] Pontes, C., et al. (2021). "Automated deployment of ELK-based SOC using IaC." *Proceedings of the IEEE Cloud Computing Conference*. ↑ Page 8

[6] Garcia, L., et al. (2022). "Fusing host and network telemetry for advanced intrusion detection." *Computers & Security*. ↑ Page 8

[7] Open Information Security Foundation (OISF). (2023). *Suricata User Guide 7.0*. https://docs.suricata.io ↑ Page 8

[8] SwiftOnSecurity. (2024). *Sysmon-config: A Sysmon configuration file for everyone*. https://github.com/SwiftOnSecurity/sysmon-config ↑ Page 8

[9] Suneja, S., et al. (2020). "Linux endpoint visibility via Auditd." *USENIX Security Symposium*. ↑ Page 8

[10] Strom, B., et al. (2018). "MITRE ATT&CK: Design and philosophy." *The MITRE Corporation*. ↑ Page 8

[11] Applebaum, A., et al. (2021). "Quantifying detection coverage with the MITRE ATT&CK framework." *IEEE Symposium on Security and Privacy*. ↑ Page 8

[12] Roth, F., & Patzke, C. (2024). *Sigma: Generic Signature Format for SIEM Systems*. https://github.com/SigmaHQ/sigma ↑ Page 8

[13] MITRE Corporation. (2024). *CALDERA: Automated Adversary Emulation Platform*. https://caldera.mitre.org/ ↑ Page 8

[14] Red Canary. (2023). *Atomic Red Team: A library of small, highly portable detection tests*. https://github.com/redcanaryco/atomic-red-team ↑ Page 8

[15] Miller, R., et al. (2022). "Empirical measurement of MTTD in modern EDR platforms." *ACM Digital Threats: Research and Practice*. ↑ Page 8

[16] Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. *Proceedings of the 2009 IEEE CISDA*. ↑ Page 21

[17] Hasan, M., Islam, R., & Rahman, M. (2021). A comparative analysis of open-source SIEM

solutions for intrusion detection. *Journal of Network and Computer Applications*, 178, 102966.

[18] Erdal, E., & Chai, S. (2022). Evaluating EDR solutions through ATT&CK-based adversary emulation. *Proceedings of IEEE S&P Workshops*, 88–97.

[19] Opitz, N., & Zimmermann, S. (2020). Infrastructure-as-code in enterprise security: A systematic review. *Computers & Security*, 96, 101865.

[20] Hussain, F., Abbas, S. G., Pires, I. M., & Hussain, F. K. (2022). A review of SOAR tools for security orchestration and automated response. *IEEE Access*, 10, 38814–38832.

[21] OWASP Foundation. (2023). *OWASP Top Ten 2023*. https://owasp.org/Top10

[22] OWASP Foundation. (2024). *OWASP Juice Shop*. https://owasp.org/www-project-juice-shop

[23] Alazab, M., & Tang, M. (2022). Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications*, 64, 103057.